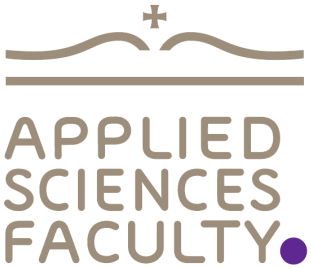
# C++ — паралельні обчислення: огляд




©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

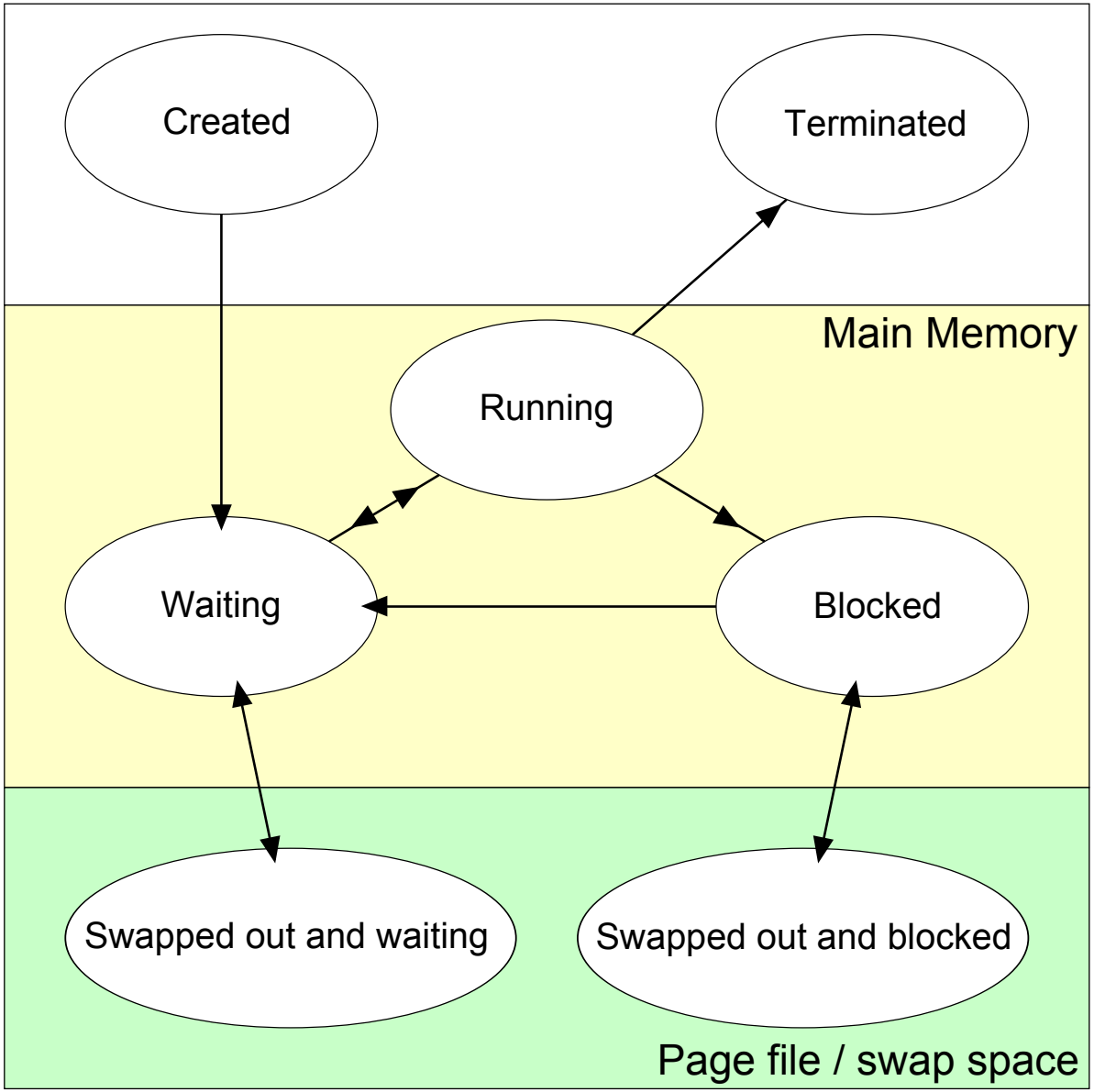
# Процеси

- Процес (process) — екземпляр комп'ютерної програми, що виконується, із відповідним набором ресурсів – регістрами, лічильником команд, адресним простором, відкритими файлами, тощо.
- Згідно Таненбаума (див. список літератури), Windows формулює це поняття трішки по іншому, більш плідно: процес – це контейнер для ресурсів, в якому міститься один або більше потоків виконання, кожен із яких має свої регістри, лічильник команд, стек, але спільно користується рештою ресурсів процесу.
- Одна і та ж програма може одночасно виконуватися в декількох процесах.
- Багатозадачність — одночасне виконання декількох процесів.

Процес включає наступні компоненти:

- Образ (image) програми — машинний код, який виконуватиметься.
- Пам'ять процесу (часто – віртуальна), яка включає код та дані процесу, його стек та купу (heap).
- Дескриптори операційної системи, пов'язані із процесом: дескриптори файлів, вікон, тощо.
- Права доступу процесу, включаючи його власника.
- Стан (контекст) процесу — в першу чергу, вміст його регістрів.
- В багатьох сучасних ОС, завдяки механізму віртуальної пам'яті, процеси знаходяться в різних "паралельних просторах". При цьому ОС надає засоби комунікації між різними процесами (IPC — Inter-process communication).

# Стани процесу




Зображення взято з Вікіпедії.

# Потоки (threads[2]) — 1

- Потік — частина процесу, яка може виконуватися паралельно із іншими його частинами. Такий собі процес-лайт.
- Кожен процес має принаймні один потік – головний потік, у якому і починає виконуватися.
- На відміну від різних процесів, потоки розділяють спільну пам'ять та деякі інші ресурси. Тобто, процес може містити декілька потоків, які є його частиною.
- Завдяки спільному контексту, переключення між потоками швидше ніж переключення між процесами.
- Потоки умовно можна поділити на потоки ядра (kernel threads) та користувацькі потоки (user threads). Різниця між ними в тому, що першими займається ядро ОС, а другі реалізовані в user-space, виключно засобами, доступними користувацьким програмам[1]. Користувацькі потоки більше ефективні, так як не потребують переключення в контекст ядра для зміни потоку. Однак, диспетчер ядра про них нічого не знає.

[1] Тобто, декілька потоків (user-space) у одному процесі можна реалізувати лише засобами C++ чи іншої мови програмування.

[2] Українською іноді перекладається як "нитка"

# Потоки — 2

- Для максимально ефективного використання наявних процесів використовується також модель M:N, гібридна – M програмних потоків, N – потоків ядра. (В таких термінах виключно ядерні потоки – модель 1:1, виключно користувацькі – N:1).
- Популярності набувають також кооперативні моделі багатозадачності в межах процесу[3] – співпрограми (couroutines) та "волокна" (fibers)[4]. Така концепція може підтримуватися як на рівні ОС[5] (Windows 7, GNU Portable Threads), так і конкретними мовами програмування[6] (вбудована підтримка є в D, Erlang, Go, Haskell, JavaScript, Lua, Perl, PHP, Prolog, Python, Ruby, Tcl та багатьох інших).
- Генератори, типу тих, що реалізовано в Python (із виразами yield) — більш обмежена концепція, ніж співпрограми.

---

[3] Все ж, комп'ютерна історія розвивається по спіралі – завжди це повторював, поки не виявив, що Таненбаум сформулював цю ідею помітно раніше, і не факт чи був першим. :-) Так і тут – від кооперативної багатозадачності ранніх Windows (аж до 3.11), до примусового витіснення, і назад до кооперативної, на іншому рівні.

[4] Див., наприклад, `https://agraphicsguynotes.com/posts/fiber_in_cpp_understanding_the_basics/`.

[5] Тоді дані супер-легкі кооперативні потоки називають fiber, волокнами.

[6] У цьому випадку швидше буде вжите слово співпрограма, coroutine. В принципі, місце реалізації — основна відмінність між цими термінами, хоча, можливі інші формулювання.

Детальніше див., наприклад:

- Ендрю Таненбаум, "Сучасні операційні системи" ("Modern Operating Systems"), видання четверте, "Глава 2. Процеси і потоки". А краще – взагалі всю першу частину.
- `https://en.wikipedia.org/wiki/Process_(computing)`
- `https://en.wikipedia.org/wiki/Thread_(computing)`
- `https://en.wikipedia.org/wiki/Fiber_(computer_science)`
- `http://en.wikipedia.org/wiki/Multithreading_(software)`
- `https://en.wikipedia.org/wiki/Coroutine`
- `https://en.wikipedia.org/wiki/Mutual_recursion`
- `https://en.wikipedia.org/wiki/Parallel_computing`

"Сирі" потоки — річ дуже низьковірнева, а паралельне програмування із їх використанням — неймовірно складне.

# Інші види паралелізму

Сучасні комп'ютери підтримують багато інших способів розпаралелювати обчислення:

- SIMD — Single instruction, multiple data. Сучасні процесори здатні опрацьовувати кілька порцій даних (наприклад, цілих чисел чи чисел із рухомою крапкою) за раз[7]. Однак, їх ефективне використання вимагає великого об'єму складної низькорівневої роботи. Тому найбільш прагматичним способом цими засобами скористатися — використовувати спеціалізовані бібліотеки, автори яких провели відповідну роботу або покладатися на автоматичну векторизацію компіляторів, яка поки, на жаль, відносно малопотужна. Дана паралельність ідейно дуже відмінна від тієї теми, що розглядається в цих лекціях.
- Сучасні відеокарти містять десятки чи й сотні простеньких, але достатньо гнучких обчислювальних "модулів" – процесорів. Існує ряд технологій для роботи із ними: CUDA, пропрієтарна від NVidia; OpenCL — більш відкрита, для написання гетерогенних програм, котрі одночасно використовують різні класи обчислювальних пристроїв-"співпроцесорів", ще один гетерогенний стандарт – OpenACC і т.д. До відеокарт тут примикають і пристрої класу Xeon Phi.
- FPGA дозволяють реалізовувати спеціалізовані процесори "в залізі".
- Ще один важливий вид паралелізації — **розподілені обчислення**, випадок, коли програма виконується одночасно на різних комп'ютерах, з'єднаних із використанням тієї чи іншої мережі.

[7] SSE для x86, NEON для ARM, AltiVec для PowerPC.

# Програмний інтерфейс

- Так як робота з процесами та потоками — річ низькорівнева, ОС надають відповідний програмний інтерфейс, API.
- Для спрощення розробки та забезпечення крос-платформовості, багато які мови містять свої примітиви роботи з потоками та процесами, які позбавляють необхідності безпосередньо працювати з ОС-специфічними API.
- До C++11 C++ не мала своїх вбудованих засобів багатопоточності. Для роботи із процесами існувало декілька примітивів, запозичених із C, такий як функція system(), для більш складних операцій слід було звертатися до POSIX-функцій, функцій WinAPI, аналогічних засобів менш популярних ОС, або сторонніх бібліотек. C++11 цю, доволі сумну, ситуацію, виправило[8].

[8] Хоча, ряд важливих абстракцій все ще не входять в стандарт.

Поміж зовнішніх, відносно мов програмування, бібліотек чи API операційних систем слід згадати:

- Pthreads — реалізацію POSIX-потоків (доступна і під Windows).
- Умовно назвемо WinThreads – реалізацію потоків у Windows.
- Сімейство POSIX-функцій fork/exec та їх Windows-аналогів CreateProcess і spawn.
- Boost.Interprocess для взаємодії між процесами.
- Boost.Thread — реалізацію потоків, яка послужила прототипом для них у C++11.
- Бібліотека Threading Building Blocks (TBB) від Intel реалізовує більш високорівневі засоби, самостійно турбуючись про маніпуляцію низькорівневими засобами.
- OpenMP — стандарт на явне розпаралелення програм за допомогою директив-підказок, який підтримується багатьма компіляторами, включаючи GCC, ICC, MSCV.
- Крім того, більшість великих GUI-toolkit-ів та framework-ів включають свої засоби роботи з багатопоточністю. Вони є і в Qt, і в wxWidgets, і в FLTK, і т.д і т.п.

# Альтернативи до багатопоточності

- Використання кількох процесів, які обмінюються повідомленнями — pipes або технології типу OpenMPI. (Shared memory має більшість основних проблеми багатопоточності, тому мова не про неї!)
- Більшість GUI API та бібліотек (тих що toolkit/framework) надають засоби дозволити обробку повідомлень під час тривалих операцій, типу Sleep у Win32API (подробиці див. `http://winapi.freetechsecrets.com/win32/WIN32Yield.htm`) та QEventLoop::processEvents() в Qt (див. відповідну лекцію).
- Засоби виклику функцій у вказані моменти часу – таймери, типу boost::asio::deadline_timer з boost чи QTimer з Qt.
- callback-и у відповідь на певні події.
- Різноманітні засоби асинхронного виконання, типу std::async.
- Високорівневі інтерфейси, типу MapReduce (`https://en.wikipedia.org/wiki/MapReduce`), наприклад **QtConcurrent**, які дозволяють розподіляти наявні ресурси між складними обчислювальними задачами, інкапсулюючи низькорівнені маніпуляції із потоками.

# Закон Амдала

- Закон стверджує, що частина програми, яка не розпаралелюється, обмежує максимальний виграш від паралелізації.
- Максимальне прискорення від паралелізації можна записати так:

$$S_{\text{latency}}(s) = \frac{1}{1 - p + \frac{p}{s}},$$

де $S_{\text{latency}}$ — потенційне (відносне) прискорення програми, s — прискорення паралельної частини завдяки розпаралеленню, p — частка програми, що піддається паралелізації. Тобто, навіть в ідеальному випадку:

$$S_{\text{latency}}(s) < \frac{1}{1 - p}$$

- Іншими словами, якщо виключно послідовна частина програми займає 10% часу виконання, скільки б не було доступних апаратних потоків виконання, не вдасться прискорити її більше ніж в 10 раз. Тієї ж теми стосується жарт, що навіть 100 жінок разом не зможуть народити дитину за місяць.
- Закон Амдала дозволяє оцінити, чи є сенс прикладати зусилля до паралелізації задачі.

# Закон Густафсона

- Зазвичай, задачі не є рівні за розміром, тобто, припущення, із якого виходить закон Амдала, не виконується.
- При чому, на практиці, із зростанням потужності зазвичай виникає потреба/бажання працювати із більшими об'ємами даних, тому паралельна частина росте швидше, ніж та, що не піддається паралелізації.
- В таких умовах закон Густафсона, який описує ситуацію, коли об'єм роботи лінійно росте із кількістю апаратно підтримуваних потоків виконання (процесорних ядер), може бути адекватнішим:

$$S_{\text{latency}}(s) = 1 - p + sp,$$

- Зрозуміло, що його оцінки — менш песимістичні. Але реальна задача, зазвичай, знаходиться десь між умовами цих законів — фіксованим об'ємом задач і лінійним ростом їх паралельної частини із зростанням кількості ресурсів.

## Залежності та паралельне сповільнення

- Жодну програму не вдасться прискорити більше, ніж найбільший час взаємо-пов'язаних обчислень.
- Після певної кількості потоків задача почне виконуватися повільніше навіть ніж однопоточна — накладні витрати на керування потоками (наприклад, переключення контекстів) та обмін інформацією між ними (будь це спільна пам'ять плюс блокування чи та або інша форма явного обміну повідомленнями) забиратимуть більше ресурсів, ніж виграш.
- Звичайно, перед цим наступить момент, коли N+1 потоків повільніше вирішують задачу, ніж N.

Перефразовуючи відому фразу, паралелізація — як член. Добре мати, добре вміти користуватися, але розмахувати без потреби не варто.
Підходьте до її використання із здоровим глуздом.

# Data race або стан гонитви

- Найбільш загально — помилка, яка виникає, коли результат операції починає залежати від послідовності та тривалості виконання окремих команд.
- Надзвичайно важкі для пошуку помилки – спроба їх локалізувати дуже легко приховує їх. Під час роботи в дебагері, після додавання логування, і т.д., вони можуть просто зникнути — через зміну таймингу подій.
- Сам термін походить із електроніки.
- В програмуванні може стосуватися гонитви при роботі із пам'яттю, файлами, мережею та іншими подібними ресурсами. Надалі говоритимемо про race condition стосовно пам'яті.
- C++11/C11 формулює так: якщо кілька потоків одночасно звертаються до тієї ж змінної, при тому принаймні один із них записує у неї, маємо гонитву даних. При цьому поведінка невизначена — undefined behavior.
- Взагалі, одна із найскладніших проблем паралельного програмування.
- Так як шукати подібні помилки дуже важко, слід зразу проектувати та писати програми належним чином!

# Способи боротьби із data race

- Писати виключно однопоточні програми. Звичайно, при тому втрачаються переваги, які надає багатопоточність.
- Використовувати блокування для кожної змінної, яка може спільно використовуватися. Як і в попередньому випадку, може звести нанівець зростання продуктивності від наявності кількох потоків. Плюс загроза deadlock і т.д.
- Акуратно вибирати, де і коли блокуємо. Підхід ефективний, але складний у реалізації та вразливий до помилок і поганого дизайну.
- Засоби автоматичного детектування. Ще б існували такі засоби із гарантією (є інструменти, які успішно детектують найпростіші випадки).
- Проектувати багатопоточну програму так, щоб потоки взаємодіяли тими чи іншими повідомленнями, не використовуючи спільних змінних. Має свої плюси та мінуси. Див. також MPI та міжпроцесну взаємодію.
- Використання високорівневих засобів, які інкапсулюють подробиці роботи із потоками.

# Atomic та lock-free

- Написання програм без явних блокувань називається lock-free програмуванням.
- Існують lock-free структури даних, (хоч їх створення і є надзвичайно складною задачею!).
- Елементом для створення таких структур та програмування без блокувань є атомарні змінні — ті, звертання до яких не може бути перервано іншим потоком.
- Звичайно, користуючись засобами блокування — мютексами і т.д., можна зробити атомарним будь-який тип даних. Однак, при тому страждатиме продуктивність.
- Всі сучасні процесори надають певний набір типів та операцій над ними, які є атомарними.
- Насправді, навіть створення мютексів без атомарних операцій було б неможливим.

# Atomic та lock-free — посилання

- http://habrahabr.ru/post/244881/ — Розповідь про атомарні операції.
- http://habrahabr.ru/post/150801/ — Немного о многопоточном программировании. Часть 1. Синхронизация зло или все-таки нет.

Бар'єри пам'яті і Lock-free структури даних

- http://habrahabr.ru/company/ifree/blog/195770/ — "Lock-free структуры данных. 1 — Начало"
- http://habrahabr.ru/company/ifree/blog/195948/ — "Lock-free структуры данных. Основы: Атомарность и атомарные примитивы"
- http://habrahabr.ru/company/ifree/blog/196548/ — "Lock-free структуры данных. Основы: откуда пошли быть барьеры памяти."
- http://habrahabr.ru/company/ifree/blog/197520/ — "Lock-free структуры данных. Основы: Модель памяти."
- http://habrahabr.ru/post/155507/ — "Демонстрация сбоев программы при отсутствии барьеров памяти"

# Засоби синхронізації (Draft!)

- Мютекси
-





# Thread-local storage

# Посилання — вікіпедія

- https://uk.wikipedia.org/wiki/Паралельні_обчислення, https://en.wikipedia.org/wiki/Parallel_computing.
- https://uk.wikipedia.org/wiki/Закон_Амдала, https://en.wikipedia.org/wiki/Amdahl's_law
- https://uk.wikipedia.org/wiki/Закон_Густавсона_—_Барсіса, https://en.wikipedia.org/wiki/Gustafson's_law
- http://en.wikipedia.org/wiki/Mutual_exclusion
- http://en.wikipedia.org/wiki/Readers–writer_lock
- http://en.wikipedia.org/wiki/Semaphore_(programming)
- http://en.wikipedia.org/wiki/Reentrant_mutex
- http://en.wikipedia.org/wiki/Lock_(computer_science) http://en.wikipedia.org/wiki/Spinlock
- http://en.wikipedia.org/wiki/Linearizability — зокрема, атомарність.
- http://en.wikipedia.org/wiki/Concurrency_control
- http://en.wikipedia.org/wiki/Thread-local_storage

Для лінивих — огляд термінології та основних нюансів багатопоточності:
http://www.bogotobogo.com/cplusplus/multithreaded.php































